Proof-Carrying Pipelines

Proof-Carrying Pipelines: Offloading CI/CD Gate Execution to Untrusted-but-Identified Machines with Cryptographic Attestations

Jack Andrés Cid *Independent work — v1.2, August 2026*

Abstract

Continuous integration pipelines re-execute the same validation gates — linters, static analyzers, compilation, unit tests, contract checks — on shared cloud runners for every push, even when the developer already ran exactly those gates locally seconds earlier. This redundancy is treated as the price of trust: the pipeline cannot know that the local run happened, ran the *current* rules, or ran them against the *pushed* content. We present **Proof-Carrying Pipelines (PCP)**, a pattern in which pipeline gates execute on untrusted-but-identified machines (typically developer workstations) and their verdict travels with the commit as a cryptographic attestation that binds together (i) the exact content hash, (ii) the digests of the validation tools, (iii) the version of the rule set, and (iv) a timestamp, signed by an organization-held key (HSM/KMS) that the machine can invoke but never possess. A lightweight **attest-and-skip gate** at the pipeline choke point verifies the attestation in seconds and elides redundant re-execution; any missing, stale, drifted, or invalid proof triggers the full pipeline as a fail-closed fallback. A freshness-and-drift lock keeps local tooling pinned to the pipeline's sources of truth, so a proof from an outdated toolchain is never accepted. PCP occupies a deliberately pragmatic point in the design space: it provides *organizational* integrity — attributable, revocable, audited execution by enrolled identities — without requiring trusted execution environments or fully reproducible builds, and degrades gracefully to classic CI when its assumptions fail. We describe the protocol, its threat model, a reference implementation, and a production-shaped case study on an enterprise data platform where contract, SQL-model and DAG validation gates were offloaded with first-push compliance preserved. The name is a deliberate homage to proof-carrying code: the artifact carries the evidence; the checker stays cheap.

---

1. Introduction

Two forces are colliding in modern software delivery. Pipelines keep absorbing more validation — style, static analysis, policy-as-code, schema and contract checks, unit tests — because centralizing them is how organizations guarantee they run. Meanwhile developer machines have become absurdly capable and sit idle during every CI wait. The result is a familiar loop: run the checks locally, push, then wait for a shared runner to run the same checks again, queued behind every other team, at cloud prices.

The waiting is not the deep problem; *trust* is. The pipeline re-executes because it cannot believe three things about a local run: that it **happened at all**, that it ran over **exactly the content that was pushed**, and that it used the **currently approved tools and rules** rather than last month's. Any mechanism that establishes those three facts more cheaply than re-execution converts redundant cloud compute into a signature check.

Culturally, the demand exists. Basecamp's `gh-signoff` (2025) lets a developer mark a commit green after running checks locally — by plain self-declaration, with no binding to content, tools, or rules. Its reception captured both halves of the situation: enthusiasm for reclaiming local execution, and immediate criticism that nothing prevents a stale toolchain, a dirty approximation of the pipeline, or a simple lie. PCP is, in one sentence, `gh-signoff` with the missing cryptography and the missing operational discipline.

Contributions.

1. A named, general pattern — *Proof-Carrying Pipelines* — for offloading pipeline gate execution to untrusted-but-identified machines under organizational key custody, with verify-and-skip semantics at the pipeline.
2. A minimal attestation binding — content hash × tool digests × ruleset version × identity × time — that is necessary and, we argue, sufficient for the elision decision.
3. Two protocol obligations that prior self-attestation approaches lack: a **fail-closed fallback** (absent/invalid proof ⇒ the full pipeline runs, so PCP can never be *less* safe than classic CI) and a **freshness-and-drift lock** (proofs from drifted tooling are rejected and the local bundle refuses to sign until it self-updates).
4. A reference implementation and a case-study instantiation on an enterprise data platform.

2. Related work

Supply-chain attestation. in-toto provides signed link metadata proving that supply-chain steps were performed by authorized functionaries, verified against a layout at the end of the chain. PCP borrows its core insight — sign the *execution of a step*, not just the artifact — but repurposes it for a different decision at a different point: elide *re-execution inside the pipeline itself*, with an explicit fallback path and an economics-driven framing (compute offload). SLSA provenance, sigstore/cosign and GitHub Artifact Attestations sign what was built and where; they document provenance rather than replace pipeline work. PCP is deliberately format-compatible with this ecosystem: the attestation payload can be exported as an in-toto Statement (DSSE envelope) under a PCP predicate type (SPEC O9), so transparency logs and policy engines can consume PCP proofs as part of the broader supply chain without understanding attest-and-skip semantics.

Hardware-rooted approaches. Attestable Builds (2025) and the evidence-driven protocol of Castillo et al. (2026) obtain stronger guarantees — attested execution state — by running build/test steps inside TEEs (e.g., Intel TDX), often combined with reproducible builds and transparency logs. These designs remove the need to trust the executing party at all. PCP is the pragmatic complement: no TEE requirement, no reproducibility requirement, deployable today on unmodified laptops, at the cost of trusting enrolled identities within an audited, revocable, fail-closed envelope. Where TEEs are available, they slot into PCP as a stronger signer.

Determinism-based trust. Tweag's untrusted-CI work and Trustix trust build *outputs* by content-addressing and cross-builder comparison under Nix. This verifies results without signatures but requires deterministic builds and addresses artifact caching rather than policy gates (many of which — linters, rule engines — are cheap to re-run but *matter* because their rule sets change).

Build caches. Bazel remote caches, Nx Cloud and Turborepo skip recomputation keyed by input hashes. The known weakness is trust in cache writers ("can you trust your build cache?"): access to the cache is authority to inject results. Signature support where it exists covers transport integrity of artifacts, not "the gates ran, with these tools, under these rules." PCP is the missing trust layer for the *skip* decision, applied to pipeline gates.

Self-attestation. `gh-signoff` demonstrates the demand and the gap simultaneously: local sign-off with no cryptographic binding. PCP's attestation is precisely the set of facts whose absence its critics pointed at.

Self-hosted CI runners. The alternative most platform teams evaluate first is not any of the above but self-hosted runners (GitHub Actions, GitLab CI): move execution onto machines you control. The two patterns solve different problems — runners relocate *where* work runs; PCP removes *redundant re-execution* and makes the resulting verdict portable — and their trust models differ in kind. A runner's "green check" is ambient

trust: a row in the forge's database, written because a registered runner process said so, bound to nothing the verifier can independently check, mutable by anyone with sufficient admin rights (including via the commit status API). PCP's verdict is an explicit, signed, content-bound, revocable claim checked by predicates at decision time.

It is instructive to ask what one would have to *add* to self-hosted runners to approximate each PCP guarantee: content/tool binding requires signed per-job provenance (in-toto or artifact attestations — i.e., adopting attestation itself); tool integrity against runner drift requires ephemeral, digest-pinned runner images plus hash-pinned actions and an admission process comparing runner images to a source of truth (a rebuilt drift lock); identity and revocation require per-job workload identity mapped to enrolled principals (runner registration tokens identify a pool, not a principal); verdict non-forgeability requires restricting workflow-file edits, branch rulesets, *and* removing status-API write paths — or, again, signing the verdict with a key the runner cannot hold; freshness requires a custom result-aging policy (forge checks never expire); fail-closed degradation requires scripted fallback pools (a job with no available runner queues rather than degrading); and audit-plus-sampling requires bespoke re-run pipelines. Assembled together, these pieces are a re-implementation of PCP's producer with forge primitives — scattered across settings that do not compose into one verifiable claim, and still rooted in the forge's mutable control plane rather than KMS custody. The practical conclusion is complementarity: run the PCP bundle *on* the self-hosted runner (the runner is a natural enrolled Producer) and keep a single cheap verifier at the choke point.

Axis	**PCP**	**Self-hosted runners**
Problem solved	Redundant re-execution	Location/control of execution resources
Trust model	Explicit, signed, revocable per identity	Ambient — implicit in infrastructure and forge control plane
Verdict	Content-bound, portable, predicate-checked	Forge DB row; not independently verifiable; never expires
Availability	Fail-closed: no proof ⇒ classic path runs automatically	No native failover; jobs queue while no runner is available
Work lost on failure	Minimal — execution and certification are decoupled	Entire job, if the runner dies mid-run (no checkpointing)
Complementarity	The runner can act as a PCP Producer	—

Proof-carrying code. Necula and Lee's PCC (1996) established the asymmetry PCP exploits: the untrusted producer does the expensive work and ships evidence; the consumer runs a cheap checker. PCP's "proofs" are attestations of gate execution rather than formal proofs of program properties — the checker verifies *who ran what over what content with which tools*, not the semantic property itself. The homage is intentional and the limitation explicit (§7).

3. Threat model

Assets. Integrity of the deployment gate decision: nothing reaches protected branches / deployment without the required gates having passed over the exact delivered content.

Adversaries considered. (A1) A *careless* developer: stale tools, dirty tree, forgot to run checks. (A2) A *pressured* developer attempting to shortcut gates without deliberately attacking cryptography. (A3) An *external* attacker who compromises the transport, the repo remote, or replays old proofs. (A4) *Drift*: the organization changes rules/tools and outdated local bundles keep signing.

Out of scope (declared, not hidden). (A5) A developer with valid signing rights who deliberately fabricates a verdict by tampering with the *local execution itself* (e.g.,

patching the validator binary before it runs) is not prevented by PCP without hardware attestation — this is the TEE boundary. PCP mitigates A5 by pinning tool *digests* (the attestation names the exact container images/binaries), by audit logging every signature (KMS/Cloud Audit), by supporting random re-verification (the pipeline re-runs a sampled fraction of skipped gates), and by making the skip *revocable per identity*. Organizations for which A5 is intolerable should combine PCP with TEE-backed signers or forgo elision.

Adaptive sampling and identity reputation. Sampled re-verification (O3) need not use a flat rate. The natural operational refinement is per-identity adaptation: a divergence found in a sampled re-run escalates that identity's sampling rate — up to 100%, which is de facto loss of elision — and can suspend its elision rights automatically pending review (O8). Reputation systems are usually dangerous to automate because the penalty destroys value on false positives; PCP's fail-closed structure makes automatic quarantine safe by construction — the penalty is only the loss of an optimization, so a false positive costs runner minutes, never safety. Trust becomes a tunable, per-identity, self-correcting quantity.

(A5′) Autonomous coding agents as producers. When the producer's workload is driven by an autonomous coding agent, a behavior functionally equivalent to A5 arises *without malicious intent*: reward hacking — the agent weakens an assert, skips a test, or edits a rule to "reach green." The cryptography is again intact; the claim is hollow. For agent-signed attestations we recommend: a *separate enrolled identity per agent* (an agent must never sign as the human who invoked it — this also makes O1 audit logs meaningful), a shorter TTL than for human producers, a substantially more aggressive sampling rate for re-verification (O3/O8), and *ephemeral execution*: unlike a human's workstation, an agent needs no persistent environment, so each agent attestation should originate from a fresh single-use container — persistent tampering with the producer environment then cannot survive across runs. Organizations can tune trust per identity class rather than per protocol. When the agent's own tests are part of the attested gate set, a mutation-score gate (Böckeler 2026) is the computational complement to these controls: hollow tests pass coverage but stop killing mutants, so reward-hacked test suites become visible *inside* the proof rather than only through sampling.

Key custody. Signing keys live in an HSM-backed KMS; machines hold *invocation rights* (IAM), never key material. Compromise of a laptop yields the ability to sign — visible in audit logs, revocable in minutes — not the key.

Preconditions the protocol does not provide. Two are worth stating explicitly. First, *pinning is identity, not hardening*: a digest guarantees you ran exactly the approved image — it says nothing about whether that image is secure. An unhardened image pins, signs and verifies flawlessly. PCP assumes the approved-tool catalog has passed hardening and scanning *outside* the protocol; this holds equally when an entire runner sandbox is collapsed into a single "golden image" digest inside the lock — the verifier stays simple, but the digest's meaning remains identity and immutability, never security posture. Second, *lock custody*: if the producer can edit `gates.yaml`/`versions.lock` before the signing step reads them, the case reduces to A5 — either the payload then names non-approved digests (caught by the verifier's approved-set predicate, fail-closed) or the producer fabricates approved digests it did not run (A5 proper, bounded by sampling/audit/revocation). As defense in depth, the lock SHOULD be generated and frozen by a platform-managed process separate from the gate execution environment, never writable from it.

4. The pattern

Roles. *Producer*: the identified machine executing gates (developer workstation, edge runner). *Signer*: org-held KMS key the producer may invoke. *Verifier*: the attest-and-skip gate in the pipeline. *Sources of truth*: the repositories/registries that define gates, tool images, and rule sets.

Protocol.

1. **Pin.** The local bundle materializes the gates from the sources of truth and records a lock: `{source SHAs, tool image digests, ruleset digest, locked_at}`.
2. **Execute.** Gates run locally — ideally in the *same container images* the pipeline uses.
3. **Bind.** On PASS, the producer canonicalizes a payload: `{content: git_tree_hash, tools: [image digests], rules: ruleset_digest, verdict, identity, timestamp}`.
4. **Sign.** The payload digest is signed via KMS (e.g., EC-P256/SHA-256). The attestation travels with the commit (git note, trailer, or sidecar).
5. **Verify (attest-and-skip gate).** The pipeline recomputes the tree hash, fetches the public key, verifies the signature, and checks four predicates: signer ∈ enrolled set; tool digests ∈ currently approved set; ruleset digest = current; age ≤ TTL.
6. **Skip or fall back.** All predicates hold ⇒ redundant gates are elided (registration, deployment and any *non-redundant* stages still run). Any predicate fails ⇒ the full pipeline executes. **Absence of proof is never an error; it is the classic path.**
7. **Drift lock.** The local bundle checks its lock against the sources of truth before signing; on drift or staleness it still validates but refuses to sign (advisory verdict) until self-updated. This makes rule rollout instantaneous: bumping the ruleset digest invalidates every outstanding proof at the verifier without coordinating with producers. For developer ergonomics the bundle should *pre-fetch* lock updates in the background (O10) so the mandatory pre-sign check rarely blocks at push time — pre-fetching moves latency off the critical path without weakening the check.

Monorepo aggregation (design sketch). In a monorepo, one commit can touch dozens of targets, each with its own gate set. Naively this means one attestation — and one KMS invocation — per target. The verifier is not the bottleneck (a signature check is milliseconds); the costs are producer-side KMS quota/latency and audit-log noise. The natural aggregation is Merkle-shaped: build one payload per affected target, arrange their digests as leaves of a Merkle tree, and sign the root *once*. The attestation carries the root signature plus, per target, its payload and inclusion path; the verifier checks the single signature and the inclusion proof for whichever targets the pipeline run actually covers. One KMS call, one audit-log entry, per-target verification granularity preserved.

Design notes. The binding must be to the *tree hash* (content), not the commit hash, so history rewrites that preserve content preserve proofs, and any content change breaks them. TTL bounds replay of “honest but old” proofs (A3/A4). Random re-verification converts A5 from undetectable to statistically detectable. The verifier must be *cheap and boring* — one signature check and four set-membership/equality tests — because its cost bounds the pattern’s benefit.

Two adoption modes, two economics. The protocol is indifferent to who the producer is; the business case is not. In *offload mode* — the framing of §1 — producers are machines already paid for and idle (developer workstations, on-prem rigs), and the saving is real displacement of billed runner minutes. In *attested-cache mode*, the producer is itself CI infrastructure (e.g., a self-hosted runner executing gates once whose verdict later pipelines skip): nothing is offloaded — the benefit is “build once, verify many” with an explicit, revocable trust model replacing cache ACLs. Both are legitimate; a platform team should not assume offload-mode economics when deploying cache-mode topology.

5. Reference implementation

The reference implementation (this repository) is ~500 lines of dependency-light Python + Bash: `pcp attest` runs declared gates (`gates.yaml`: command + pinned image digest each) in their containers, canonicalizes the payload (RFC 8785-style JSON), and signs via pluggable backends (local Ed25519 for the demo; Google Cloud KMS asymmetric-sign in production); `pcp verify` implements the attest-and-skip predicate

set and emits an exit code suitable for `allow_failure: false` CI jobs; a `versions.lock` + `pcp update` pair implements the drift lock. A 30-line GitHub Actions / GitLab CI snippet shows the gate wiring with fallback.

6. Case study (anonymized)

On a large retail enterprise's self-service data platform, every push re-ran four gate families on shared runners: SQL-model rule validation (18 org rules), DAG static analysis, contract structure validation, and dbt compile+unit tests. We instantiated PCP with the platform's own validator containers and deployed rule documents as sources of truth. The local bundle reproduced all five CI gates bit-for-bit against a generated pipeline (same pinned dbt-core, same rule JSONs), reached PASS parity with the cloud pipeline on first push, and the drift lock correctly downgraded verdicts to advisory when the lock aged past 14 days or upstream SHAs moved. Wall-clock effect: the redundant portion of the pipeline (minutes of container startup, dependency resolution and re-validation on shared runners) collapses to a signature verification (seconds); compute shifts to machines already paid for and idle. The fail-closed property held trivially: proofs absent ⇒ the untouched classic pipeline ran.

7. Security analysis and limitations

PCP's guarantee is *organizational*, not *physical*: "these gates passed over exactly this content, with exactly the approved tools and rules, executed under an enrolled, audited, revocable identity, recently" — it is not "no party could have cheated." The elision decision inherits the strength of (a) key custody (HSM-grade), (b) identity enrollment (IAM-grade), (c) tool pinning (registry-grade), and (d) the audit/re-verification regime. A5 (malicious enrolled producer) is bounded, not eliminated: pinned digests force the attacker to tamper *around* the tools rather than with them; sampling re-verification bounds the expected survival of fabricated verdicts; audit logs make every signature attributable. Gates whose outcomes depend on environment (integration tests against live services) are poor PCP candidates and should stay in the pipeline; PCP targets *hermetic* gates (lint, static analysis, policy checks, compilation, unit tests), which in practice dominate redundant CI time. Finally, PCP does not accelerate cold-start consumers (a fresh clone still runs everything); its economics target the high-frequency inner loop.

Secrets in local execution. Some gates need credentials — typically read access to private package registries during dependency resolution. Two-layer answer. First, a truly hermetic gate should need *no* live secrets: vendored dependencies or a read-only registry proxy remove the need entirely, and a gate that requires credentials to live services is failing the hermeticity test and should not be elided in the first place. Second, for the legitimate residue (private package pulls), apply *minimal injection*: short-lived, read-only, narrowly scoped credentials issued against the producer's enrolled identity — the same IAM surface that already grants KMS invocation — never long-lived organizational secrets stored on the machine. A compromised producer then leaks at most a scoped, expiring, revocable read token, and the audit trail already knows which identity held it.

Availability. Because execution and certification are decoupled, signer unavailability is not a new failure mode — it is the existing one. If the KMS is unreachable, the producer cannot sign; an unsigned push carries no proof; and absence of proof *is* the classic path: the full pipeline runs, automatically, with zero configuration. Nothing stalls and nothing is lost — the locally executed gate work remains valid and can be signed once the signer returns, within the TTL. The only cost of a signer outage is forgone elision for the pushes it covers. This degrades favorably against self-hosted runner infrastructure, where an unavailable runner leaves jobs queued and a runner dying mid-job loses the entire job's work (no native checkpointing). Fail-closed is thus PCP's availability story as much as its security story: every partial failure — no proof, stale proof, invalid proof, unreachable signer — converges to the same safe default, classic CI.

Maturity prerequisites. PCP is not a starting point; it presupposes a platform that has already invested in reproducible, governable CI. Before adopting, an organization should have: (1) *containerized gates* pulled by digest from a controlled registry — if gates run on ad-hoc toolchains there is nothing bindable to pin; (2) *versioned sources of truth* for rule sets and tool catalogs, so `rules` and `tools` digests have an authoritative referent and rollout is a digest bump (O4); (3) *KMS/HSM key custody with per-identity IAM invocation and audit logging* — if keys live on laptops the trust claims collapse; (4) an *identity enrollment and revocation process* (joiner/leaver discipline for producers, human or agent); (5) a *programmable pipeline choke point* able to verify-then-elide with a fail-closed default; (6) *gate telemetry* (per-stage durations) to identify which gates are hermetic and redundant and to size the benefit honestly; (7) an *owner for the operational regime* — someone accountable for the approved set, sampling rate (O3/O8) and revocations; and (8) a *scoped short-lived credential path* for the gates that legitimately need private-registry access (see Secrets above). Teams missing (1)–(3) should treat those as the roadmap: they are valuable independently of PCP, and PCP is then a small increment on top. Conversely, an organization with none of this maturity gains little from the pattern and should not start here.

Validation status. This is independent work that has not yet undergone peer review. The evidence base is one anonymized industrial case study; the reference implementation is small (~500 lines) and has had no external security audit. None of this changes the pattern's logic, but readers evaluating adoption in regulated environments should calibrate accordingly — and treat the sampling, audit and revocation obligations (O1–O4) as load-bearing, not optional.

8. Future work

TEE-backed signers as a drop-in producer upgrade; transparency-log countersigning of attestations (sigstore/Rekor) for third-party auditability; implementing the Merkle aggregation sketched in §4 and the in-toto/DSSE export (O9); formalizing the verifier predicates; and measuring fleet-level compute displacement at scale.

References

- Necula, G. C., Lee, P. *Safe Kernel Extensions Without Run-Time Checking* / Necula, G. C. *Proof-Carrying Code*, POPL 1997.
- Torres-Arias, S. et al. *in-toto: Providing farm-to-table guarantees for bits and bytes*, USENIX Security 2019.
- SLSA — Supply-chain Levels for Software Artifacts. https://slsa.dev
- Sigstore. https://sigstore.dev · GitHub Artifact Attestations, 2024.
- *Attestable builds: compiling verifiable binaries on untrusted systems using trusted execution environments*, arXiv:2505.02521, 2025.
- Castillo, F., Brito, E., Pullonen-Raudvere, P., Werner, S., Tai, S. *An Evidence-driven Protocol for Trustworthy CI Pipelines*, arXiv:2605.21089, 2026.
- Tweag. *Untrusted CI: automatic trusted caching of untrusted builds with Nix*, 2019. · Trustix, nix-community.
- Nx. *Can You Trust Your Build Cache?* (blog). · Turborepo remote caching + artifact signature verification (docs).
- Basecamp. *gh-signoff* — https://github.com/basecamp/gh-signoff, 2025.
- Böckeler, B. *Maintainability Sensors for Coding Agents*, martinfowler.com, 2026 — esp. "The Test Suite as a Regression Sensor".